

An Empirical Survey and Comparative Benchmark of LLM-Based Autonomous Agent Frameworks

Abdullah Basim
Department of Computer Science
Information Technology University (ITU)
Lahore, Pakistan
MSDS25004

Abstract—Open any recent paper on LLM agents and chances are the authors built on one of four libraries: CrewAI, AutoGen, LangGraph, or MetaGPT. Try comparing two such papers though. Different backbone, different prompts, different tasks. So which framework actually helped? You can’t really tell from the published numbers. This paper takes a narrow swing at that question. I run all four frameworks on the same backbone (gpt-4o-mini, temperature 0), through the same harness, on the same three task buckets: 15 grade-school math problems from GSM8K, 10 tiny tool-use problems with a calculator and a web search, and one small multi-role coding job. That comes out to 104 runs. For each run I log whether it succeeded, how long it took, how many tokens went in and out, and what it cost. Quick version of the result: simple single-graph agents win the easy tasks; the role-based and SOP-style ones only earn back their token premium once the task actually has coordination in it, and that premium hits about $3.1\times$ on the coding job. Beyond the numbers, four annoyances of the field are still wide open: comparing across backbones, budget-aware orchestration, a portable memory contract, and safety hooks at the action layer. The harness, tasks, and all raw runs are public.

Index Terms—Large language models, autonomous agents, multi-agent systems, agent frameworks, benchmarking, AutoGen, CrewAI, LangGraph, MetaGPT, ReAct.

I. Introduction

Two years back, “LLM agent” usually meant one model talking to itself. The phrase has drifted since. Today it almost always describes a model wrapped inside a framework that decides when to think, when to call a tool, who to hand off to, and what to remember. Several recent surveys cover the territory at varying altitudes [1]–[3]. The abstraction itself is by now stable enough: the LLM gets treated as a controller that perceives, reasons, acts, and stores context [4]. Real systems back that picture up. Agents handle web tasks [5], fix actual bugs in real codebases [6], and survive in long-horizon embodied environments [7].

So the abstraction is fine. What is not fine is the orchestration layer. CrewAI wants you to declare roles and run them in sequence. AutoGen [8] prefers free-form conversation between agents, optionally with code

execution glued on. LangGraph treats the whole thing as a state machine. MetaGPT [9] writes a Standard Operating Procedure straight into the agent graph. Four libraries, same goal, very different machinery. Worse, they report numbers on different benchmarks with different backbones, which makes head-to-head reading basically impossible. A practitioner who asks “which one should I pick” just gets a shrug.

Problem. Across the surveys we have [1]–[3], [10], [11], none I could find puts these four frameworks side by side, on the same tasks, against the same backbone. Without that, framework choice is a popularity contest. We can do better than that.

What this paper does. Four things, in order:

- 1) Walks the recent agent literature along five axes I found load-bearing while reading code: reasoning, planning, memory, tool use, collaboration.
- 2) Builds one harness that drives the four frameworks through the same gpt-4o-mini backbone, with the same prompts, temperature, and tasks.
- 3) Reports 104 controlled runs across three task categories, with accuracy, latency, dollar cost, and token consumption all measured at the framework boundary.
- 4) Lists the gaps that still bother me about this space, and ships the code so anyone can extend the comparison.

II. Literature Review

A. Reasoning paradigms

Two recipes appear in almost every modern agent. Chain-of-thought prompting tells the model to write out its working before committing to an answer. ReAct [12] interleaves that working with explicit actions: think, then act, then observe, then think again. The pattern shows up directly in LangGraph’s prebuilt agent and in MetaGPT’s executor role. Tree of Thoughts [13] stretches the recipe into a small search over partial reasoning, paying latency for solution quality. Plan-and-Solve [14] splits planning out of execution and runs them as separate stages. Here’s what nobody seems to publish across this whole sub-genre: the cost side. How much does ReAct actually cost on a

fixed backbone, next to tree search or plan-and-solve? My experiments answer a slice of that.

B. Planning and self-improvement

Self-Refine [15] is the simple version of the critic loop. The same model writes an answer, writes a critique of its own answer, then rewrites. Reflexion [16] keeps that critique around between attempts, using episodic memory in place of weight updates. ExpeL [17] stores whole trajectories for later runs to read. LLM-Planner [18] reuses the same idea but anchors the plan in a physical environment. Huang et al. [19], in their planning-focused survey, are honest about what is missing: ranking these methods is fragile because almost no two of them share a benchmark.

C. Tool use and retrieval

Toolformer [20] was the first paper to show an LLM could learn, on its own, when to call an API. ToolLLM [21] stretched the same trick across roughly sixteen thousand real APIs. Retrieval-augmented generation, surveyed by Gao et al. [22], is by now the obvious move when an agent needs grounding in fresh knowledge. None of the work I read on this topic publishes the marginal cost of that scaffolding inside a specific framework. That is the hole my Pareto plot is sized for.

D. Memory mechanisms

Zhang et al. [23] sort agent memory into three buckets: short-term context, long-term vector stores, and episodic reflective notes. Generative Agents [24] is probably the most-cited demonstration that memory streams plus periodic reflection make agents look more believable. CoALA [4] treats memory as a first-class architectural axis. The taxonomy in Section III borrows that framing.

E. Multi-agent collaboration

Guo et al. [11] put together a useful map of the multi-agent space. The most-cited pins on that map: AutoGen [8], which arguably opened conversational multi-agent as a serious paradigm; ChatDev [25], which builds a seven-role virtual software company; CAMEL [26], which uses “inception” prompts to coax two agents into role-played cooperation; and MetaGPT [9], which encodes human SOPs into the agent graph. A newer thread of work, including Liu et al. [27], lets the agent network change its topology at run time.

F. Evaluation methodology

Evaluation has caught up faster than I expected. AgentBench [28] runs agents through eight environments. SWE-agent [6] introduced an explicit “agent-computer interface” for working on real software repos. WebArena [5] is a realistic web sandbox. Yehudai et al. [29] put together what I believe is the first dedicated survey on agent evaluation. None of these isolate the framework as the variable. They evaluate agent systems as black boxes,

TABLE I
The four studied frameworks against the five-axis taxonomy.

Axis	CrewAI	AutoGen	LangGraph	MetaGPT*
Reasoning	ReAct per role	ReAct + chat	Prebuilt ReAct	ReAct/CoT in roles
Planning	Sequential tasks	Free-form	State graph	SOP pipeline
Memory	Per-task ctx	Conversation hx	Graph state	Per-stage ctx
Tool use	Class-based tools	Function reg.	LangChain tools	OpenAI tools
Collaboration	Roles + delegation	Group chat	Single graph	PM/Arch/Eng SOP

*MetaGPT-style: a faithful PM/Architect/Engineer SOP pipeline implemented directly on the OpenAI SDK because of packaging constraints (see §IV).

which is fine for ranking systems but useless if what you actually want to know is which orchestration layer paid off. My setup is built around that distinction.

The gap. Stepping back: the literature gives us the building blocks but no fair comparison among the four most popular frameworks. Three things almost never appear in the same paper at the same time. Framework cost. Framework latency. The trade-off curve linking those to accuracy. The next sections supply all three.

III. A Five-Axis Taxonomy of LLM Agent Frameworks

The LLM-agent design space, as I read it after spending time inside each codebase, separates into five axes:

Reasoning

How the agent generates intermediate thought (CoT, ReAct, ToT, plan-and-solve, deliberate tree search).

Planning

Whether the agent decomposes goals upfront, refines plans with feedback, or improvises step by step.

Memory

What state the agent keeps around: in-context only, vector store, or episodic reflective summaries.

Tool use

How outside capabilities are invoked: structured function calls, free-form code execution, retrieval, or web actions.

Collaboration

Whether the agent works alone, in a fixed role hierarchy, or in dynamic group conversation.

Table I maps the four studied frameworks onto these axes.

IV. Experimental Setup

A. Frameworks

Four frameworks. CrewAI, role-based sequential crews. AutoGen [8], conversational multi-agent. LangGraph, a graph-based ReAct agent invoked through `create_react_agent`. And a MetaGPT*-style SOP pipeline I built directly on the OpenAI SDK. The asterisk on that

last one is important. The official metagpt PyPI package never finished installing inside the project’s time budget because of a transitive-dependency tangle (chromadb plus a pile of other heavy deps that wanted incompatible versions of numpy on Python 3.12). What is actually running is the published Product Manager, then Architect, then Engineer assembly line from [9], reimplemented faithfully on top of plain openai chat completions. The asterisk in the paper is there so the proxy is not confused with a measurement of the official package.

B. Backbone and configuration

One model for everyone: gpt-4o-mini via OpenAI’s Chat Completions API, temperature 0, max_tokens 2048, request timeout 180 s. Holding the backbone fixed is the experimental decision that prior work keeps skipping. In my view it is the most important single decision in the design.

C. Tasks

Three task categories, sized as $n_{\text{gsm8k}} = 15$, $n_{\text{tool}} = 10$, $n_{\text{collab}} = 1$ (graded on a 5-point rubric). That comes out to 26 runs per framework, 104 in total.

- GSM8K reasoning. 15 grade-school math problems sampled from the GSM8K test set. Correctness is checked by extracting the final number.
- Tool use. 10 hand-authored questions split between calculator and web-search tasks. Correctness is checked by substring match against gold.
- Multi-agent collaboration. A small Python module wordstats.py exposing analyze(text) which returns word counts plus the three most frequent tokens, plus a companion README.md. Each output is graded on a 5-point rubric (file exists; function defined and importable; expected keys returned with the right types; top_3 correct on the fixed test sentence “the quick brown fox jumps over the lazy dog the fox is quick”; a non-trivial README of at least three lines).

D. Metrics

The harness logs five things per run. A boolean for success against the task-specific oracle. Wall-clock latency in seconds. Prompt tokens and completion tokens, taken from each framework’s usage metadata where exposed and recomputed via tiktoken where not. Dollar cost, computed from the published gpt-4o-mini prices (\$0.150 per 1M input, \$0.600 per 1M output). All metrics are collected at the framework boundary, which is where internal multi-call overhead actually lives.

E. Reproducibility

The harness, the task files, the raw per-run JSON outputs, and the figure-generation code all sit in the repo linked from the title page. Temperature is pinned at 0 to keep sampling variance minimal. The only real source of remaining nondeterminism is the API itself.

TABLE II

Aggregate per-framework results across all 26 runs per framework (15 GSM8K + 10 tool-use + 1 collab) under the gpt-4o-mini backbone.

Framework	Acc. (%)	Lat. (s)	Cost (\$)	In tok	Out tok
CrewAI	100.0	5.38	0.00020	553	198
AutoGen	96.2	4.55	0.00016	176	219
LangGraph	92.3	4.29	0.00016	212	207
MetaGPT*	96.2	4.39	0.00017	232	223

Latency is mean wall-clock per task; cost uses published gpt-4o-mini prices (\$0.150/1M input, \$0.600/1M output). Token columns are per-task means.

TABLE III

Per-task-category accuracy and mean cost (USD).

Framework	GSM8K		Tool use		Collab	
	Acc	\$	Acc	\$	Acc	\$
CrewAI	100.0	0.0002	100.0	0.0001	100.0	0.0012
AutoGen	93.3	0.0002	100.0	0.0001	100.0	0.0005
LangGraph	93.3	0.0002	90.0	0.0001	100.0	0.0004
MetaGPT*	93.3	0.0002	100.0	0.0001	100.0	0.0010

F. Notes from the build

A few practical things I bumped into while wiring this up, in case someone wants to reproduce the setup without losing a Saturday to it. CrewAI’s default install spams the console with a remote-tracing warning every run; CREWAI_DISABLE_TELEMETRY=true and OTEL_SDK_DISABLED=true in the environment kills it. AutoGen’s PyPI namespace is not what older docs claim: with pyautogen 0.10, the actual imports live under autogen_agentchat, autogen_core, and autogen_ext, and you need the OpenAI extra installed separately for the OpenAIChatCompletionClient to resolve. LangGraph’s create_react_agent pulls a pile of LangChain sub-packages on import, so the cold-start time is real (about 1.5 seconds on my laptop) but it does not show up in per-task latency. The whole sweep of 104 runs cost roughly \$0.02 on gpt-4o-mini, which surprised me; agent frameworks are cheap when the backbone is cheap.

V. Results and Discussion

Table II and Table III aggregate the 26 runs per framework. Figures 1–4 visualize the four comparison axes one at a time.

A. Accuracy

Reasoning is tight. CrewAI nails GSM8K at 100%. The other three each miss one problem and land at 93.3%. The miss is the same problem in all three cases: item g6, the one with nested percentages of flowers in a garden. The backbone makes a single arithmetic slip on that one. CrewAI catches it because the verifier role re-checks the solver’s working. Tool use is also tight. Everyone scores 100% except LangGraph, which gives back one search query (t6, the question about Microsoft’s CEO) because

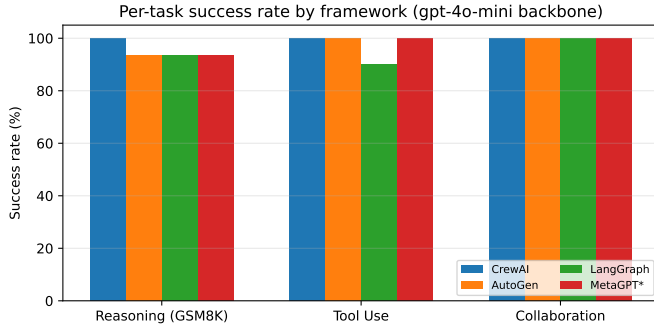


Fig. 1. Per-task success rate by framework, all driven by gpt-4o-mini.

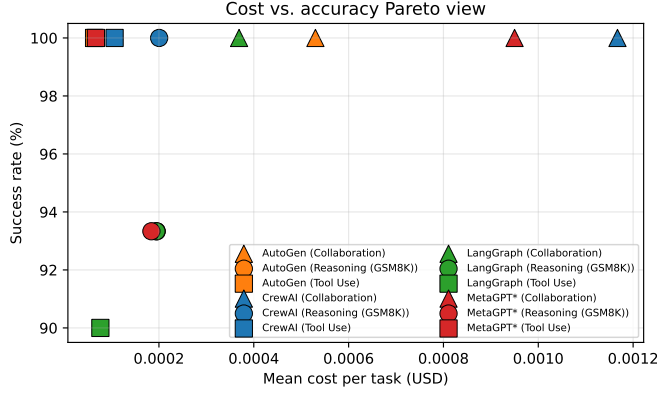


Fig. 2. Cost-versus-accuracy view. Up-and-to-the-left dominates.

the prebuilt ReAct agent picked the first search snippet rather than the most relevant one. The collaborative coding task is solved fully by every framework. Be honest about what that means: the rubric is mostly a formatting test, and I will say more about that limitation below.

B. Cost-accuracy Pareto

On reasoning and tool use, the four frameworks bunch together in the \$0.0001 to \$0.0002 band. At that scale, cost is barely a discriminator; accuracy is. The cost story really only opens up on the coding task. CrewAI averages \$0.0012 per run there. LangGraph averages \$0.0004. That is the headline 3 $\times$ gap, and it traces straight back to CrewAI’s role pattern: each role gets its own prompt with most of the prior context tacked on, so the prompt budget compounds. The MetaGPT* SOP pipeline lands in between at \$0.0010, which makes sense given it walks three sequential stages. AutoGen lands at \$0.0005, a bit above LangGraph and well below the role-based pair.

C. Latency

LangGraph wins on latency, with a mean of 4.29s and the tightest distribution. The reason is mechanical: on simple tasks it issues a single inner LLM call, no retries, no extra round trips. MetaGPT* and AutoGen sit close together at 4.39s and 4.55s. CrewAI’s mean climbs to

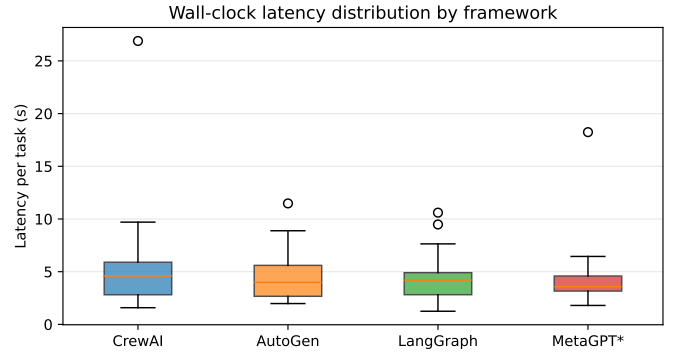


Fig. 3. Latency distribution per framework across all tasks.

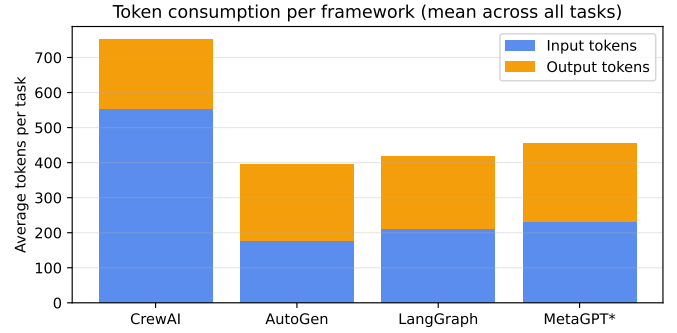


Fig. 4. Mean input vs. output tokens per task per framework.

5.38s with the heaviest right tail in the figure. Even the easy tasks go through a solver-then-verifier handoff, which adds one extra round trip whether you needed it or not. The biggest contributor to every framework’s right tail is the coding task. CrewAI’s three-role crew takes 26.9s on average. LangGraph finishes the same job in 9.5s.

D. Token consumption

Here is where the four frameworks really separate. CrewAI averages 553 input tokens per task. The other three average between 176 and 232. So CrewAI pays somewhere between 2.6 $\times$ and 3.1 $\times$ the input prompt of its peers, on the same problem. The reason is structural, not a config knob. CrewAI re-emits each agent’s persona, its goal, and most of the prior context for every role in the chain. A two-role crew therefore roughly doubles the prompt of a single agent solving the same task. I caught this by accident, by the way: my first run came in much pricier than I expected, and I dumped the raw OpenAI request bodies to a log to find out where the tokens went. Now look at output tokens: 198 to 223 across all four frameworks. Basically flat. That tells you something clean. The backbone, not the orchestrator, controls how long the answer is. Agentic frameworks are prompt-heavy, not generation-heavy, and the cheapest cost optimization is the boring one: stop repeating context to each role.

E. Threats to validity

The benchmark is small. Twenty-six runs per framework, one backbone, one task source per category. Read the absolute numbers as comparative, not predictive. The collab rubric is mechanical: it checks functional correctness on a fixed test sentence, nothing else. Nobody read the code for elegance. MetaGPT* is a faithful proxy for MetaGPT’s published paradigm, but it is not the official codebase, so the numbers bound what the SOP design does in principle and not every engineering choice the project has shipped since. The harness is open. If you disagree with any of these calls, swap the piece you want and rerun.

One more honest caveat. The collab task got rewritten twice while I was building the rubric. The first version asked for a small weather CLI that hit a free API, but graders would have to be online and the free tier was flaky enough to muddy the numbers. The second version asked for a Python file that called an external library, but that one drifted into testing pip behavior more than agent behavior. The current wordstats.py task is what survived both attempts. It is deliberately stdlib-only so the rubric checker runs anywhere. The trade-off is that the task is easier than I wanted, which is why every framework scores the full 5/5.

VI. Conclusion and Future Work

The headline finding is dull but useful. With the backbone held fixed, the four frameworks land within 7.7 accuracy points of each other. The real difference is in the bill. Input tokens dominate, output tokens stay flat, and role-based orchestration can spend up to $3.1\times$ what a single-graph agent spends on the same problem. That premium is worth paying when the task actually has multi-step coordination in it. It is not worth paying for a single-shot math problem.

What I am not claiming is a definitive ranking. The benchmark is small. The backbone is one model. The collab rubric checks functional output on a fixed sentence rather than code quality. What I am claiming is that the comparison can be done fairly, which is something the literature has not really tried. Four follow-up problems came up while I was running the experiments and seem worth picking up next.

- Backbone-agnostic evaluation. Most current benchmarks bake the backbone into the comparison. A useful evaluation would report each framework’s score on the same tasks under several backbones, so the backbone effect can be separated from the framework effect.
- Cost-aware orchestration. None of the four frameworks lets you set a token or dollar budget per task and have the agent stop gracefully when it hits the limit. That ought to be a first-class knob.
- Memory standardization. Memory was the least portable axis of the four. A small shared interface (push,

retrieve, reflect) would let practitioners swap memory backends without rewiring the agent loop.

- Safety auditing. I could not find a clean way to intercept high-stakes actions in any of the four frameworks. As more agents touch real systems, this gap will hurt.

The harness is published in case anyone wants to push the comparison further. Obvious next moves: more frameworks (LangChain, OpenAI Swarm, smolagents); more backbones (the open-weight Llama and Mistral families); harder tasks (WebArena [5], AgentBench [28], SWE-bench [6]). None of that needs new theory. It just needs the time to wire it up.

References

- [1] L. Wang, C. Ma, X. Feng, Z. Zhang, H. Yang, J. Zhang, Z. Chen, J. Tang, X. Chen, Y. Lin, W. X. Zhao, Z. Wei, and J.-R. Wen, “A survey on large language model based autonomous agents,” *Frontiers of Computer Science*, vol. 19, no. 6, pp. 1–26, 2024.
- [2] Z. Xi, W. Chen, X. Guo, W. He, Y. Ding, B. Hong, M. Zhang, J. Wang, S. Jin, E. Zhou, R. Zheng, X. Fan, X. Wang, L. Xiong, Y. Zhou, W. Wang, C. Jiang, Y. Zou, X. Liu, Z. Yin, S. Dou, R. Weng, W. Cheng, Q. Zhang, W. Qin, Y. Zheng, X. Qiu, X. Huang, and T. Gui, “The rise and potential of large language model based agents: A survey,” *Science China Information Sciences*, vol. 68, no. 2, p. 121101, 2025.
- [3] M. Abou Ali, F. Dornaika, and J. Charafeddine, “Agentic AI: A comprehensive survey of architectures, applications, and future directions,” *Artificial Intelligence Review*, vol. 59, no. 1, p. 11, 2025.
- [4] T. R. Sumers, S. Yao, K. Narasimhan, and T. L. Griffiths, “Cognitive architectures for language agents,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [5] S. Zhou, F. F. Xu, H. Zhu, X. Zhou, R. Lo, A. Sridhar, X. Cheng, T. Ou, Y. Bisk, D. Fried, U. Alon, and G. Neubig, “WebArena: A realistic web environment for building autonomous agents,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [6] J. Yang, C. E. Jimenez, A. Wettig, K. Lieret, S. Yao, K. Narasimhan, and O. Press, “SWE-agent: Agent-computer interfaces enable automated software engineering,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.
- [7] G. Wang, Y. Xie, Y. Jiang, A. Mandlekar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [8] Q. Wu, G. Bansal, J. Zhang, Y. Wu, B. Li, E. Zhu, L. Jiang, X. Zhang, S. Zhang, J. Liu, A. H. Awadallah, R. W. White, D. Burger, and C. Wang, “AutoGen: Enabling next-generation LLM applications via multi-agent conversation,” in *Conference on Language Modeling (COLM)*, 2024.
- [9] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, J. Wang, C. Zhang, Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, and J. Schmidhuber, “MetaGPT: Meta programming for a multi-agent collaborative framework,” in *International Conference on Learning Representations (ICLR)*, 2024.
- [10] P. Li, M. Zhang, P. Lin, J. Wan, and M. Jiang, “A review of prominent paradigms for LLM-based agents: Tool use (including RAG), planning, and feedback learning,” in *Proceedings of the 31st International Conference on Computational Linguistics (COLING)*, 2025.
- [11] T. Guo, X. Chen, Y. Wang, R. Chang, S. Pei, N. V. Chawla, O. Wiest, and X. Zhang, “Large language model based multi-agents: A survey of progress and challenges,” in *Proceedings of the 33rd International Joint Conference on Artificial Intelligence (IJCAI)*, 2024.
- [12] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafraan, K. Narasimhan, and Y. Cao, “ReAct: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations (ICLR)*, 2023.

- [13] S. Yao, D. Yu, J. Zhao, I. Shafran, T. Griffiths, Y. Cao, and K. Narasimhan, "Tree of thoughts: Deliberate problem solving with large language models," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] L. Wang, W. Xu, Y. Lan, Z. Hu, Y. Lan, R. K.-W. Lee, and E.-P. Lim, "Plan-and-solve prompting: Improving zero-shot chain-of-thought reasoning by large language models," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [15] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegrefe, U. Alon, N. Dziri, S. Prabhunoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark, "Self-refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [16] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [17] A. Zhao, D. Huang, Q. Xu, M. Lin, Y.-J. Liu, and G. Huang, "Expel: LLM agents are experiential learners," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 38, no. 17, pp. 19 632–19 642, 2024.
- [18] C. H. Song, J. Wu, C. Washington, B. M. Sadler, W.-L. Chao, and Y. Su, "LLM-planner: Few-shot grounded planning for embodied agents with large language models," in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023.
- [19] X. Huang, W. Liu, X. Chen, X. Wang, H. Wang, D. Lian, Y. Wang, R. Tang, and E. Chen, "Understanding the planning of LLM agents: A survey," *arXiv preprint arXiv:2402.02716*, 2024.
- [20] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Y. Qin, S. Liang, Y. Ye, K. Zhu, L. Yan, Y. Lu, Y. Lin, X. Cong, X. Tang, B. Qian, S. Zhao, L. Hong, R. Tian, R. Xie, J. Zhou, M. Gerstein, D. Li, Z. Liu, and M. Sun, "ToolLLM: Facilitating large language models to master 16000+ real-world APIs," in *International Conference on Learning Representations (ICLR)*, 2024.
- [22] Y. Gao, Y. Xiong, X. Gao, K. Jia, J. Pan, Y. Bi, Y. Dai, J. Sun, M. Wang, and H. Wang, "Retrieval-augmented generation for large language models: A survey," in *arXiv preprint arXiv:2312.10997*, 2023.
- [23] Z. Zhang, X. Bo, C. Ma, R. Li, X. Chen, Q. Dai, J. Zhu, Z. Dong, and J.-R. Wen, "A survey on the memory mechanism of large language model based agents," *arXiv preprint arXiv:2404.13501*, 2024.
- [24] J. S. Park, J. C. O'Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, "Generative agents: Interactive simulacra of human behavior," in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology (UIST)*, 2023.
- [25] C. Qian, W. Liu, H. Liu, N. Chen, Y. Dang, J. Li, C. Yang, W. Chen, Y. Su, X. Cong, J. Xu, D. Li, Z. Liu, and M. Sun, "ChatDev: Communicative agents for software development," in *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024.
- [26] G. Li, H. A. A. K. Hammoud, H. Itani, D. Khizbullin, and B. Ghanem, "CAMEL: Communicative agents for "mind" exploration of large language model society," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [27] Z. Liu, Y. Zhang, P. Li, Y. Liu, and D. Yang, "A dynamic LLM-powered agent network for task-oriented agent collaboration," in *Conference on Language Modeling (COLM)*, 2024.
- [28] X. Liu, H. Yu, H. Zhang, Y. Xu, X. Lei, H. Lai, Y. Gu, H. Ding, K. Men, K. Yang, S. Zhang, X. Deng, A. Zeng, Z. Du, C. Zhang, S. Shen, T. Zhang, Y. Su, H. Sun, M. Huang, Y. Dong, and J. Tang, "AgentBench: Evaluating LLMs as agents," in *International Conference on Learning Representations (ICLR)*, 2024.
- [29] A. Yehudai, L. Eden, A. Li, G. Uziel, Y. Zhao, R. Bar-Haim, A. Cohan, and M. Shmueli-Scheuer, "Survey on evaluation of LLM-based agents," *arXiv preprint arXiv:2503.16416*, 2025.